

ENGINEERING KEYNOTE · 2026

The Verification Gap in AI Agents

Agents act confidently. We need to know when they're wrong.

Agents don't crash.
They hallucinate
and keep going.

Traditional software panics. Agents produce
plausible output — regardless of correctness.

What do agents actually get wrong?

FACTUAL

Hallucinated facts, fabricated citations, confident claims with no grounding

ACTION

Wrong tool selected, correct tool with wrong parameters, misread tool output

COMPOUNDING

Each step builds on a flawed prior step — errors amplify across a multi-step pipeline

These categories aren't independent. Compounding is what makes the other two dangerous at scale.

Compounding: one bad step multiplies



With **90% accuracy per step**, a 5-step pipeline reaches **~59% end-to-end accuracy**. The longer the chain, the higher the blast radius.

The error doesn't announce itself at step 1. It propagates silently — and takes irreversible action at step 4.

THE CORE PROBLEM



The Verification Gap

The distance between what an agent
believes it did and what it *actually*
did.

What agents report vs. what happened

AGENT OUTPUT

"I searched the customer database and found 3 matching records."

"The API call succeeded."

"I updated the record as requested."

WHAT ACTUALLY HAPPENED

Tool returned an error. Agent interpreted the empty response as zero results — and proceeded.

Rate-limited. Agent read the 429 body as a success payload.

Updated the wrong record ID — parameters hallucinated from context.

Three verification strategies engineers can ship

1

ASSERTION LAYERS

Validate structured output schemas after every tool call. Fail fast before the result propagates downstream.

2

HUMAN-IN-THE-LOOP CHECKPOINTS

Gate irreversible, high-blast-radius actions on human confirmation. Automate everything else freely.

3

AUDIT TRAILS

Log inputs, tool calls with parameters, responses, and actions taken. Make failures diagnosable after the fact.

Assertion layers: catch the gap at the seam

```
from pydantic import BaseModel, ValidationError
from typing import Any

class SearchResult(BaseModel):
    records: list[dict[str, Any]]
    total_count: int
    query_echoed: str          # verify the tool heard what we asked

def verified_search(agent_tool_call) → SearchResult:
    raw = call_search_tool(agent_tool_call)
    try:
        result = SearchResult.model_validate(raw)
    except ValidationError as e:
        raise AgentVerificationError("search tool output failed schema") from e
    return result              # only reaches downstream if valid
```

Pattern: **call** → **validate** → **proceed or escalate**. This is defensive programming applied to agent outputs — not new ideas, new targets.

When to put a human in the loop — and where

AUTOMATE FREELY

Reversible · Low blast radius

Read ops, drafts, lookups, summaries

MANDATORY CHECKPOINT

Irreversible · High blast radius

Sends, deletes, payments, publishes

Middle ground: set a configurable threshold — reversibility score + estimated blast radius → auto-approve or escalate. Make the threshold explicit in your agent's configuration, not buried in code.

Audit trails: the post-mortem you'll need

Without a trace, debugging an agent failure is guesswork. Log everything at the decision boundaries:

- Prompt sent (verbatim, with system context)
- Tools called — name and exact parameters
- Tool responses received (full payload)

- Model output (raw, before parsing)
- Final action taken — what, when, on what
- Verification result at each assertion layer

This is standard observability practice. Apply structured logging, trace IDs, and retention policies — the same discipline as a distributed service.

THE ASK

Close the gap —
before your agent
closes it for you.

Before shipping an agent to production, ask one
question:

"How will I know when this is wrong?"

Assertion layers

Checkpoints

Audit trails